

- on the serial monitor in a new line
- 17. `delay()`: To add wait time in processing in milliseconds
- 18. `setup()`: The standard function in Arduino to be only called once
- 19. `loop()`: The standard function in Arduino to be called repeatedly
- 20. `if .... Else`: Used for conditional statements
- 21. `//`: Used for adding single line comments
- 22. `/* */`: Used for adding multi-line comments
- 23. `#define`: Used for defining a constant value
- 24. `#include`: Used for including external libraries

Structuring the Arduino code

Arduino code should ideally be structured in three parts, i.e., external libraries, constants and variables, and functions. Each part follows the other. We start by calling the necessary external libraries for our project. Next, we define any necessary constants or variables required. This is followed with the required functions.

There are two standard functions that must be called in every program, i.e., `setup()` and `loop()`. The `setup()` function is used for initialising the code while the `loop()` runs certain methods for a certain number of iterations depending upon your requirements.

Shared below is an example of what an ideal code structure should look like. We shall output the message 'Hello Internet of Things' on the Serial Monitor window every 3000 milliseconds.

```
// External Libraries
#include <SPI.h>
// Constants and Variables
char text = "Hello Internet of Things";
// Custom function(s)
void sayHello()
{
    Serial.println(text);
}
// Standard functions
void setup()
```

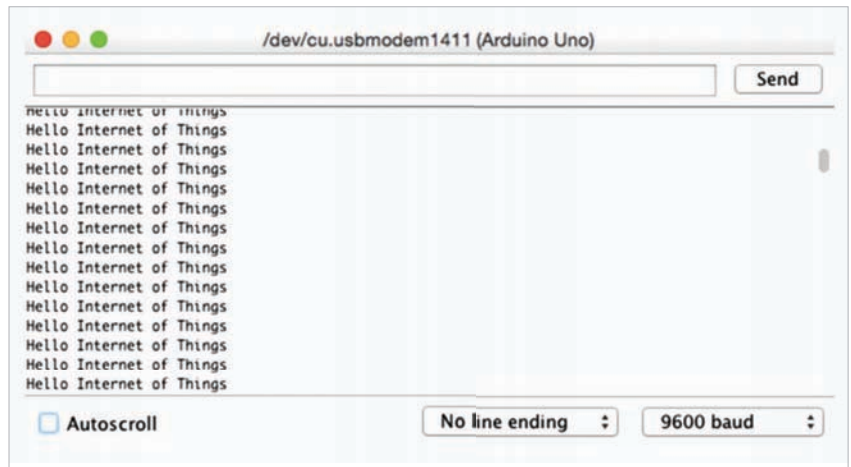


Figure 3: Output of *sayHello* program

```
{
    Serial.begin(9600); // For
    initializing the serial port
}
void loop()
{
    sayHello();
    delay(3000);
}
```

Executing the Arduino code

Now that we have the code written and structured in the proper order, we shall start the execution process. In order to run the code, you will need to do the following.

Step 1: Connect your Arduino device to the computer using the USB cable.

Step 2: In your IDE, select the Arduino board that you will be using by clicking *Tools > Board*.

Step 3: Next, you will be selecting the port by clicking on *Tools > Port*. Select the port that corresponds to the USB modem of your Arduino device.

Step 4: Click on the *Upload* button and open the Serial Monitor

window. Figure 3 shows the output on this window.

Learning to use an Arduino device can be a rewarding experience. Whether you end up liking IoT or not, just the feel of using an Arduino will give you a taste of programming hardware from the ground up. The greatest challenge that beginners face with Arduino is not the code as much as the debugging. Many times, newbie developers write code without any unit testing. My recommendation if you are just getting started with Arduino is not to get frustrated if your programs don't run properly. It can be challenging at first to find out where you may have gone wrong with your code or circuits. But over time, you will learn to create checks for your code at every level. Once you get a hang of that, the sky's the limit! Every day, people are coming up with new and interesting ways of using an Arduino device in their DIY projects. I look forward to seeing what you do with it. **END** 🐧

 By: Mir H.S. Quadri

The author is a research analyst with a specialisation in artificial intelligence and machine learning. He is the founder of Arkinfo, which focuses on the research and development of tech products using new age technologies. He shares a deep love for the analysis of technological trends and understanding their implications. Being a FOSS enthusiast, he has contributed to several open source projects.